CS233 - HW4

Nils Astrup Toft & Sondre Rogde

May 2026

Problem 1

Problem 1.a

Chamfer distance not true distance metric: From the problem statement we have that the Chamfer distance is given by

$$d_{\text{Chamfer}}(\mathbf{A}, \mathbf{B}) = (\sum_{x_B \in \mathbf{B}} \min_{x_A \in \mathbf{A}} |x_B - x_A|^2) + (\sum_{x_A \in \mathbf{A}} \min_{x_B \in \mathbf{B}} |x_B - x_A|^2)$$

We assume the norm above is the L2-norm. For this to be a true distance metric we must have that the triangle inequality must hold, i.e. $d_{\text{Chamfer}}(\mathbf{A}, \mathbf{B}) + d_{\text{Chamfer}}(\mathbf{B}, \mathbf{C}) \geq d_{\text{Chamfer}}(\mathbf{A}, \mathbf{C})$. However, a counter example is $A = \{(-1, 0, 0)\}, B = \{(0, 0, 0)\}, C = \{(1, 0, 0)\}$. Then the Chamfer distance is:

$$\begin{aligned} d_{\text{Chamfer}}(\mathbf{A}, \mathbf{B}) &= (0 - (-1))^2 + (0 - (-1))^2 \\ &= 2 \\ d_{\text{Chamfer}}(\mathbf{B}, \mathbf{C}) &= (1 - 0)^2 + (1 - 0)^2 \\ &= 2 \\ d_{\text{Chamfer}}(\mathbf{A}, \mathbf{C}) &= (1 - (-1))^2 + (1 - (-1))^2 \\ &= 8 \end{aligned}$$

We therefore have:

$$d_{\text{Chamfer}}(\mathbf{A}, \mathbf{B}) + d_{\text{Chamfer}}(\mathbf{B}, \mathbf{C}) = 4 \not\geq 8 = d_{\text{Chamfer}}(\mathbf{A}, \mathbf{C})$$

The triangle inequality doesn't hold and d_{Chamfer} cannot be a true distance metric. Observe that the same counterexample works in all dimensions by removing or adding zeros to A, B, and C above.

Deriving the gradient Let $a_k \in A$ be arbitrary. We differentiate the Chamfer distance defined above with respect to a_k and then aggregate all the derivatives into the gradient $\nabla_A d_{\text{Chamfer}}(A, B)$. Let $T_1 = \sum_{x_B \in B} \min_{x_A \in A} |x_B - x_A|^2$

and $T_2 = \sum_{x_A \in A} \min_{x_B \in B} |x_B - x_A|^2$. We differentiate each term separately, starting with T_1. If $a_k \neq \text{argmin}_{x_A \in A}|x_B - x_A|^2$ it is clear that the derivative is zero because the term would not be dependent on a_k. However, if $a_k = \text{argmin}_{x_A \in A}|x_B - x_A|^2$, then the derivative is either not defined, at the point of discontinuity, or the derivative of this term is equal to $2(a_k - x_B)$. Hence we get that

$$\frac{\partial T_1(A,B)}{\partial a_k} = 2 \sum_{x_B \in B, a_k = \text{argmin}_{x_A \in A}|x_B - x_A|^2} (a_k - x_B).$$

For T_2 we get that that only one term in the sum is dependent on a_k and hence that

$$\frac{\partial T_2(A,B)}{\partial a_k} = 2(a_k - \text{argmin}_{x_B \in B}|x_B - a_k|^2).$$

Adding T_1 and T_2 we see that

$$\frac{\partial d_{\text{Champfer}}(A,B)}{\partial a_k} = 2 \sum_{x_B \in B, a_k = \text{argmin}_{x_A \in A}|x_B - x_A|^2} (a_k - x_B) + 2(a_k - \text{argmin}_{x_B \in B}|x_B - a_k|^2).$$

We obtain the gradient $\nabla_A d_{\text{Champfer}}(A,B)$ by stacking these partial derivatives on top of each other. We are done.

Problem 1.b

The global feature is invariant to the ordering of the input points for two reasons. First, the network processes points with *shared, per-point* operations: the same MLP (implemented as 1×1 convolutions) is applied independently to each point, so permuting the input points only permutes the rows of the resulting per-point feature matrix without changing their values (permutation *equivariance*). Second, these per-point features are aggregated by max-pooling, a *symmetric* function for which order is irrelevant (e.g. $\max(x_1, x_2) = \max(x_2, x_1)$); applying a symmetric aggregator to the (re-ordered) rows yields the same global vector regardless of the input order, giving permutation *invariance*. Two other functions that preserve this invariance are min, mean and sum. Finally, if max-pooling is applied on features of dimension k, at most k of the input points can affect the final feature: each of the k output coordinates takes its value from the single point attaining the maximum in that channel, so in the extreme each coordinate is supplied by a different point, giving k contributing points.

Problem 1.c

See code

Problem 1.d

The best validation loss was achieved at epoch 392 (or 391 using 0-indexing), i.e. essentially at the very end of training. This is, however, somewhat misleading: as Figure 1 shows, the validation and test reconstruction losses drop steeply over the first $\sim$50 epochs and then flatten into a noisy plateau from roughly epoch 150–200 onwards. The apparent "best" at epoch 391 is just a small downward fluctuation on top of this plateau rather than a meaningful improvement.

Did the training loss improve after that epoch? Yes. The training loss decreases almost monotonically over the *entire* run and keeps inching downwards even in the tail, whereas the validation and test losses stop improving once the plateau sets in. This growing gap between the (still-decreasing) training curve and the (flat) validation/test curves is the classic signature of mild over-fitting: the additional epochs fit training-specific detail that does not transfer to unseen shapes.

How many epochs could we avoid? Because the test reconstruction loss is essentially flat after $\sim$200 epochs, we could stop training at around epoch 200 and lose almost nothing on the test set. In other words, roughly half of the 400 epochs could be skipped without significantly hurting the test reconstruction.

Trends. The training loss is almost monotonically decreasing and quite smooth, while the validation and (especially) test losses are noisier. All three splits decrease sharply in the early epochs; afterwards the training loss continues to creep down while the validation and test losses level off, with the test loss sitting slightly above the validation loss throughout.

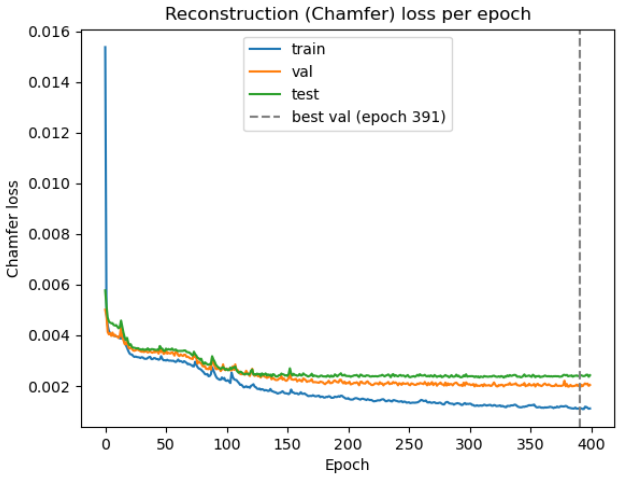


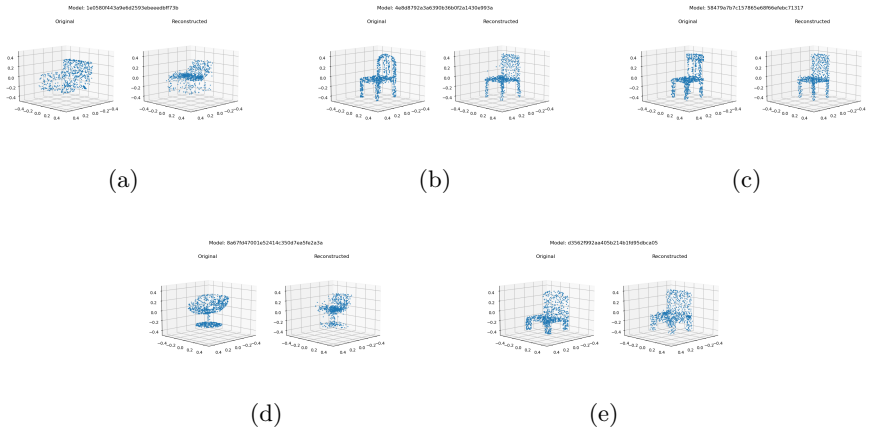
Figure 1: Reconstruction (Chamfer) loss per epoch for the train, validation and test splits of the vanilla AE. The dashed line marks the epoch of best validation loss.

Problem 1.e

Figure 2 shows the input point clouds and their reconstructions for the five test shapes. The autoencoder reliably recovers the *global* structure of each object: the seat, back, legs and overall proportions are all in the right place. There is, however, a clear and *systematic* error: thin and fine-grained structures are consistently smoothed over and blurred into diffuse clouds of points. For example, the vertical slats and the open gap of the back (Figure 2c), the rounded/arched back (Figure 2b), and the flat disc-shaped top and thin pedestal of the stool (Figure 2d) all lose some details in geometry and are reconstructed as fuzzy regions. This is expected: the decoder emits a fixed-size set of points and the Chamfer distance is content to spread them roughly over the surface, so high-frequency detail and sharp planar structures are the first thing to be sacrificed.

The best and worst reconstructions are shown in Figure 3. The best-reconstructed shape achieved a Chamfer distance of 0.00071, while the worst achieved 0.0152 - about 21× larger. The best case (Figure 3a) is a common, plain chair whose geometry is close to the "average" chair the decoder has learned, so it is reconstructed almost perfectly. The worst case (Figure 3b) is an unusually *tall* chair with a high, narrow back; the autoencoder collapses it into a normal-height chair and discards the extra height entirely. This is precisely why its loss is so much higher: atypical shapes that lie far from the bulk of the training distribution are "regressed" towards a more generic chair, so large portions of the point cloud end up far from their nearest neighbor and the squared distances accumulate.

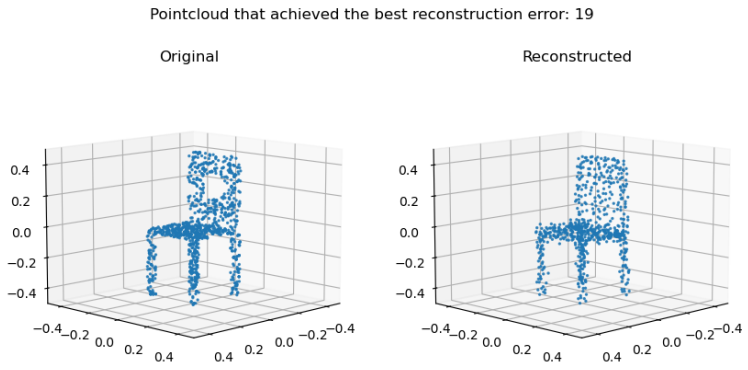
The 2D t-SNE embedding of the test-set latent codes in Figure 4 looks very natural: nearby points correspond to visually similar chairs, with recognizable groups such as office/swivel chairs, sofa-like seats, tall chairs, and small standard chairs. Even though the encoder is trained only for reconstruction, its latent space already organizes the shapes by their overall geometry, which is why the neighborhoods are semantically coherent.



(a) (b) (c)

(d) (e)

Figure 2: Input point clouds (left of each pair) and their reconstructions (right) for the five test shapes.



(a) Best reconstruction.

(b) Worst reconstruction.

Figure 3: Best and worst reconstructions from the model.

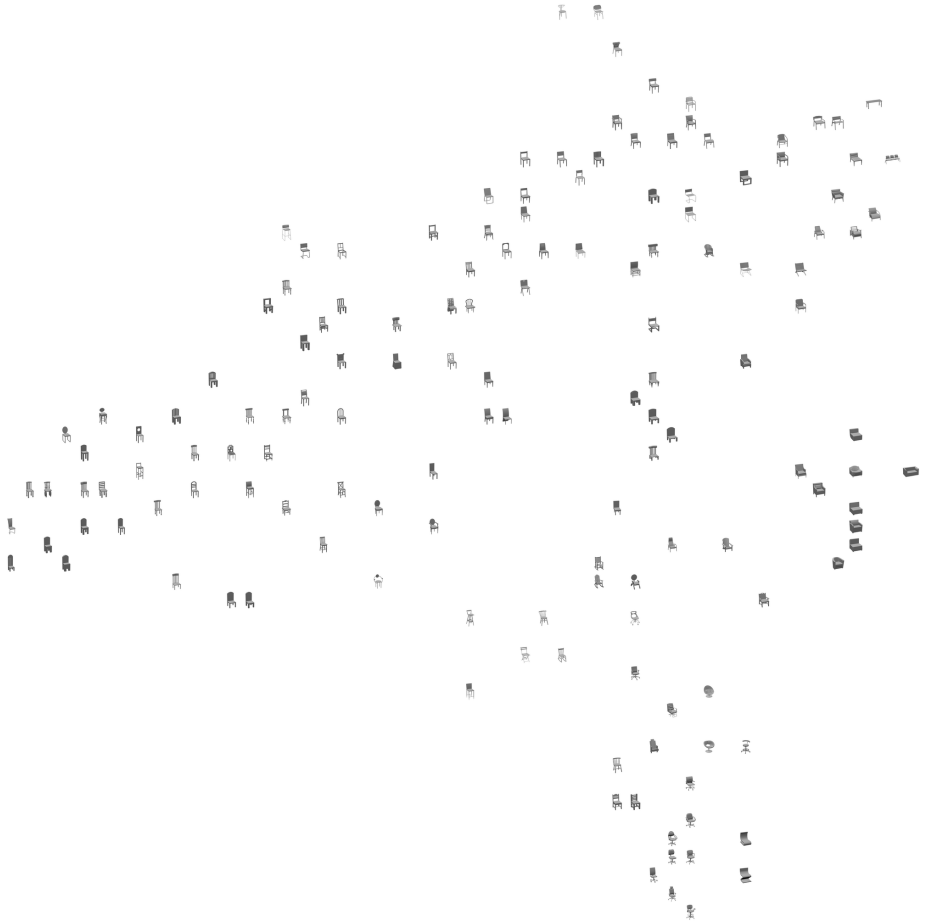
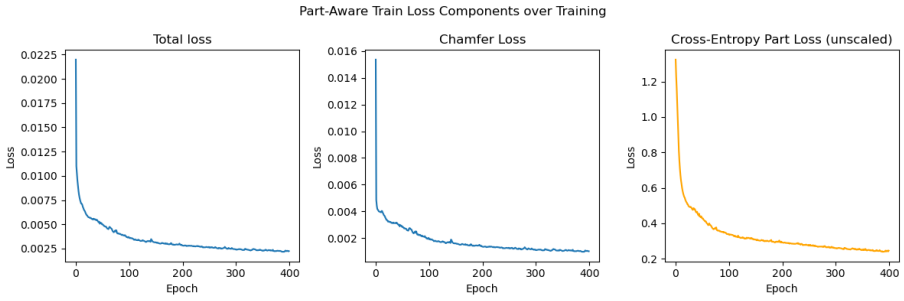


Figure 4: 2D t-SNE embedding of the test-set latent codes of the vanilla AE, with each point shown as the corresponding rendered chair.

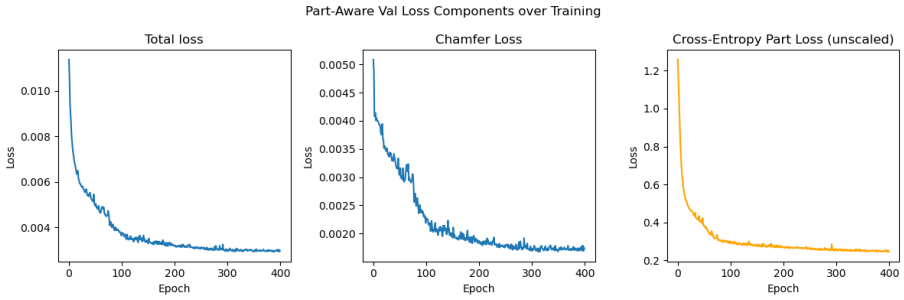
Problem 1.f

Figure 5 shows the total joint loss broken down into its Chamfer (reconstruction) and cross-entropy (segmentation) components, for both the training and validation splits. Both components decrease steeply over the first epochs and then flatten, mirroring the behavior of the vanilla AE in (d).

Did the reconstruction quality improve compared to (d)? Yes, slightly. Evaluating the optimal model (selected on the joint validation loss) on the test set, the average Chamfer distance drops from 0.00244 for the vanilla AE of (d) to 0.00207 for the part-aware AE - a reduction of about 15%. The best and worst test reconstructions improve as well (from 0.00071/0.0152 to 0.00064/0.0113). The reason is that the segmentation loss acts as an additional, structured training signal on the latent code: in order to predict the part each point belongs to, the encoder must organize its representation around the object's part composition, and this extra regularization modestly benefits the reconstruction as well. The average, per-point part-segmentation accuracy on the test set is 88.82%.



(a) Training loss components.



(b) Validation loss components.

Figure 5: Training and validation loss components per epoch.

Which task generalizes better? The segmentation task generalizes noticeably better than the reconstruction task. In Figure 5 the cross-entropy curve

is almost identical on the training and validation splits (both settling around 0.24), and the model reaches a high 88.82% per-point accuracy on the unseen test set – i.e. there is very little generalization gap. The Chamfer reconstruction, in contrast, shows a clear gap: the training Chamfer keeps decreasing to about 0.0011 while the validation/test Chamfer plateaus higher (around 0.0018–0.0021), the same mild over-fitting we observed for the vanilla AE in (d). Hence the per-point classification is the task that learns and generalizes more reliably.

Figure 6 shows, for the same five test shapes as in (e), the input point clouds colored by their *ground-truth* part labels (left of each pair) alongside the reconstruction colored by the AE's *predicted* part labels (right). The predicted segmentation is clean and spatially coherent: the back, seat, legs and (where present) arm-rests are recovered as contiguous, correctly-colored regions that match the ground truth, with errors confined mostly to the boundaries between neighboring parts (e.g. where the legs meet the seat). This is fully consistent with the ∼89% per-point accuracy, and the coloring makes it easy to see that the reconstructions preserve the object's part structure.

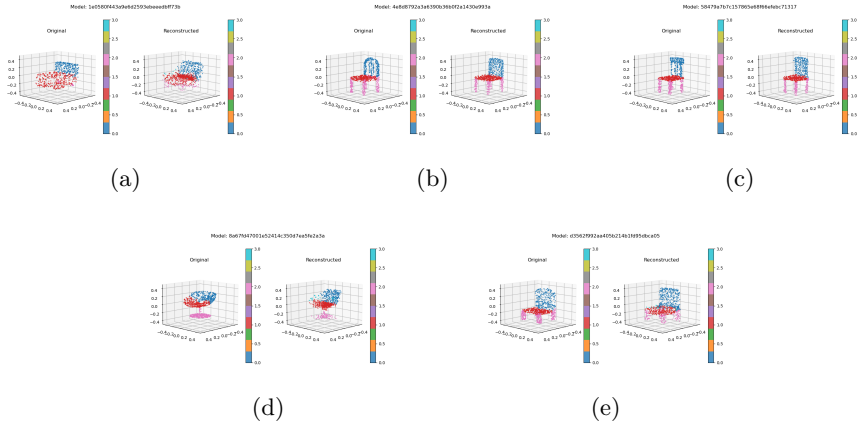


Figure 6: Part-aware model: input point clouds colored by ground-truth parts (left of each pair) and reconstructions colored by the predicted parts (right). Colors encode the part label (back, seat, legs, arm-rests).

Problem 1.g

Setup. For each chair A in the test split we find its nearest neighbor B in the *encoding* space, using the Euclidean distance between the 128-dimensional latent vectors (self excluded). We then measure how well the geometry of the latent space respects the *semantic parts* of the chairs by accumulating, over all test chairs, the one-way part-based distance of each chair A from its matched neighbor B:

$$D(A,B) = \sum_{k \in M(A) \cap M(B)} d_P\big(\tilde{M}(A,k), \tilde{M}(B,k)\big) \; + \; \max_{u} \sum_{k \in M(A) \setminus M(B)} d_P\big(\tilde{M}(A,k), \tilde{M}(B,u)\big),$$

where $M(A)$ is the set of semantic part types present in chair A and $\tilde{M}(A,k)$ is its part of type k. The first term compares the part types the two chairs *share*, like-for-like, via the golden part-distance d_P. The second term handles the parts that A has but B lacks: each such part is charged the distance to the single part $u \in M(B)$ that *maximizes* the total (a worst-case penalty, with the $\max_u$ taken outside the sum so one u is used for all unmatched parts). We report this cumulative distance, the average number of shared part types $|M(A) \cap M(B)|$, and the average latent Euclidean distance between a chair and its matched neighbor, for both the vanilla AE of (d) and the part-aware AE of (f).

Results. Table 1 reports the three metrics for both encodings, and Figure 7 visualizes them side by side.

Metric	Vanilla AE (d)	Part-aware AE (f)
Cumulative part-based distance	470.93	441.43
Avg. part-based distance per chair	3.14	2.94
Avg. # shared part types	3.10	3.13
Avg. latent NN Euclidean distance	0.17	0.28

Table 1: Part-awareness of the two encoding spaces, measured over the 150 test chairs.

Discussion. The part-aware embedding of (f) yields a *lower* cumulative part-based distance than the vanilla embedding of (d) (441.4 vs. 470.9, a reduction of about 6%; Table 1, Figure 7). In other words, when chairs are placed next to their latent nearest neighbor, the matched chairs in the part-aware space have semantic parts that are, on average, more similar to one another than in the vanilla space. This is the expected effect of the auxiliary segmentation loss: by forcing the encoder to produce a latent vector from which each point's part can be predicted, the latent code is encouraged to organize chairs according to their part composition and not just their global silhouette. The average number of shared part types is essentially the same for both models (3.10 vs. 3.13), so the

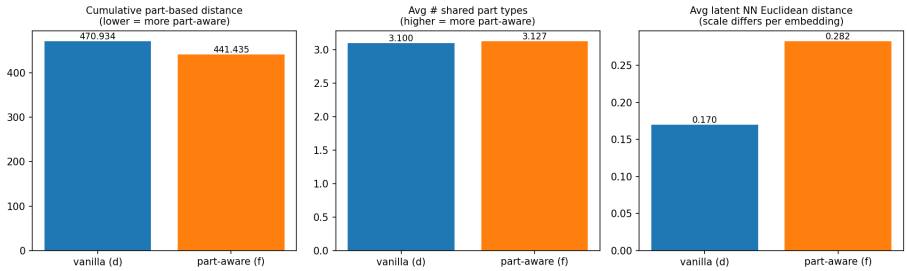


Figure 7: Comparison of the three part-awareness metrics for the vanilla (d) and part-aware (f) embeddings.

improvement comes almost entirely from *better-aligned* shared parts rather than from matching chairs that happen to have more parts in common. Finally, the average latent NN distance is larger for the part-aware model (0.28 vs. 0.17), but this is not directly comparable between the two models: the two latent spaces are unnormalized and live at different scales, so the raw Euclidean magnitude carries no part-awareness information. We therefore base the conclusion on the (scale-free) golden part-distance, which favors the part-aware encoding. Overall the effect is consistent but modest, which is expected given the small weight ($\lambda = 0.005$) placed on the segmentation loss.